

Hybrid Quantum-Classical Algorithms for Efficient Path Planning

Tuhin Hossain^{a,b,*}, Asif Aktab Ronggon^c, Mohammad Shorif Uddin^a, Mohammad Ali Moni^{d,e,**}

^a*Department of Computer Science and Engineering, Jahangirnagar University, Dhaka, 1342, Bangladesh*

^b*Department of Computer Science and Engineering, University of Scholars, Dhaka, 1213, Bangladesh*

^c*Department of Electrical and Electronic Engineering, Bangladesh University of Engineering and Technology, Dhaka, 1205, Bangladesh*

^d*AI & Digital Health Technology, Rural Health Research Institute, Charles Sturt University, Orange, NSW 2800, Australia*

^e*School of Health and Rehabilitation Sciences, The University of Queensland, Brisbane, QLD 4072, Australia*

Abstract

Efficient path planning for unmanned vehicles in complex, semi-structured environments remains a formidable challenge, primarily because of vast planning spaces and intricate environmental constraints. Classical algorithms, such as A*, often struggle with large search spaces and local optima, which limit their effectiveness in dynamic scenarios such as evacuation planning. One promising approach to overcome these challenges is to integrate quantum techniques into a traditional system to boost the performance of pathfinding algorithms. In this study, we used quantum methods to study robot navigation. The simulation experiment used 20x20 and 30x30 grids to create a two-dimensional plane space with obstacle densities of 17%, 30%, and 35%. Next, we generated a heuristic function that combines quantum computing with the classical-A* algorithm. This approach suite includes quantum models, such as the Quantum Approximate Optimization Algorithm (QAOA) and the Continuous Quantum Random Walk (CQRW), which outperform traditional search methods. The experimental results showed that CQRW-A* achieved the shortest path lengths on a 20x20 grid with a 17% obstacle density, a 30x30 grid with a 17% obstacle density, a 30x30 grid with a 30% obstacle density, and a 30x30 grid with a 35% obstacle density, yielding path lengths of 26.10, 40.70, 46.29, and 39.30 units, respectively. In contrast, the 20x20 grid showed that QAOA-A* generated the shortest path with obstacle densities of 30% and 35%, yielding (24.34 ± 1.29) and (28.52 ± 0.87) units. Finally, the experiment concluded that quantum approaches like CQRW-A* and QAOA-A* are promising for pathfinding performance over execution time.

Keywords: Quantum Approximate Optimization Algorithm (QAOA), Continuous Quantum Random Walk (CQRW), Classical-A*, Quantum Heuristics, Hybrid Quantum-Classical Algorithms, Global Path Planning.

*Tuhin Hossain

**Mohammad Ali Moni

Email addresses: tuhin97.hossain@gmail.com (Tuhin Hossain), m.moni@uq.edu.au (Mohammad Ali Moni)

1. Introduction

Path planning has recently emerged as a prominent issue, particularly in the context of robot navigation and evacuation [1]. This remains a critical challenge in robotics. Fundamental to motion planning are two concepts: motion refers to movement from one place to another, and planning refers to the formulation of a strategy to accomplish the intended motion. In motion planning, a robot follows a predefined path through a known environment to achieve a specific goal [2, 3]. The coverage path planning (CPP) issue is determining the best, most efficient, and safest way to traverse a given region.

Despite advancements in classical path-planning techniques, traditional algorithms such as A* face significant limitations in dynamic, large-scale environments, where the search space is vast and computational costs increase exponentially. These algorithms often become trapped in local optima, rendering them suboptimal in real-world scenarios in which the environment frequently changes. By contrast, quantum algorithms exploit the principles of superposition and entanglement, allowing them to explore multiple solution paths concurrently. This can provide a significant advantage in overcoming local minima and computational bottlenecks, enabling more efficient path optimization in complex environments.

Although quantum computing holds great promise, it is still in its early stages and faces scalability challenges. Therefore, hybrid quantum-classical approaches have emerged as a practical solution, leveraging the robustness of classical algorithms to provide a quantum advantage for specific tasks. In the context of path planning, hybridizing quantum heuristics with classical search strategies yields a scalable, optimized, and robust solution that navigates complex, obstacle-dense environments while leveraging quantum speedup in global optimization.

Robot path planning optimization differs from conventional path planning optimization. Intelligent optimization algorithms, particularly heuristic methods [4], are crucial for path planning in dynamic environments. Particularly prominent algorithms for path planning include the bee colony algorithm [5], ant colony algorithm [6], genetic algorithm [7], A* algorithm [8], particle swarm optimization algorithm [9], and artificial potential field algorithm [10]. Using optimization features such as minimum energy consumption, shortest time, and shortest distance, the main goal of path planning is to map the safest and most efficient route from the origin to the destination [11, 12, 13]. Recent technological advances have led to the application of path-planning approaches based on quantum optimization algorithms and deep learning [14, 15, 16]. To accelerate computations within a classical algorithm, a new area of research called hybrid (quantum-classical) algorithms combines a central processing unit (CPU) with a quantum processing unit (QPU) [17].

Quantum optimization methods utilize quantum bits (qubits) and quantum gates [18] to enhance solution sets by simultaneously exploring multiple possibilities through quantum superposition and interference. These quantum properties enable algorithms to navigate complex solution spaces more efficiently, resulting in faster computation and more reliable global optimality detection than traditional evolutionary algorithms. Quantum Genetic Algorithms (QGA) and Quantum Approximate Optimization Algorithms (QAOA) have been widely applied in fields such as production scheduling, data mining, and combinatorial optimization, demonstrating their broad utility and theoretical significance.

In this study, we demonstrate the implementation of an approach that combines classical robotics

with quantum computing (QC). We investigated the motion challenges of robots in a quantum decision-making setting. The production rules inform the quantum rule-processing algorithm’s processing of the robot’s activity planning in a recognize-act cycle. Systems equipped with artificial intelligence (AI) can benefit significantly from this method’s ability to boost computational performance. The interconnected nature of quantum theory and human reasoning, particularly its multitasking capabilities, motivated the deployment of quantum algorithms in this system.

The main contributions of this study are as follows:

- i. We present a novel hybrid framework that integrates QAOA and CQRW-based heuristics into a classical-A* search, enabling efficient navigation through highly complex, obstacle-rich environments characterized by dense polygonal obstacles and large configuration spaces.
- ii. We rigorously designed and mathematically formulated a custom cost Hamiltonian for the QAOA that incorporates obstacle proximity and path cost, along with a weighted Laplacian-based continuous quantum walk model that captures global spatial connectivity, allowing the heuristic to reflect both local and nonlocal environmental structures.
- iii. We experimentally prove that these quantum heuristics maintain admissibility and consistency, ensuring the preservation of A*’s optimality and completeness guarantees, even in high-dimensional, cluttered search spaces, thereby facilitating more informed and globally aware search strategies in challenging robotic navigation scenarios.
- iv. Through extensive experiments on 20x20 and 30x30 grids with varying obstacle densities, we demonstrate that the proposed quantum algorithms, QAOA-A* and CQRW-A*, consistently outperform classical-A* in path optimization, highlighting their potential for real-world applications.

In Section 2, we offer a summary of the planned study, and in Section 3, the methodology is detailed. In Section 4, the environment setup, experimental findings, and performance evaluation metrics for the experimental configuration are described. Finally, Section 5 recommends future research and discusses its limitations.

2. Related Work

This section presents numerous approaches for robot path planning, representing a prospective generalization of traditional and quantum path-planning methodologies.

2.1. Traditional Path Planning Approach

Liu et al. [19] introduced an EGA for robot path planning in complex scenarios. Using adaptive crossover and mutation probabilities and a reference population, the system avoids obstacles and optimises paths. Compared to typical genetic algorithms, EGA reduced path length by 20%, planning time by 25%, and success rate by 98% in complex contexts. The Improved Ant Colony Algorithm (IACA), developed by Li et al. [20], addresses traditional ACA difficulties including delayed convergence, local optima, and long pathways in mobile robot path planning. For search efficiency and

robustness, the technique improves pheromone initialisation, updating, and volatilisation rules. In testing on 20x20 and 30x30 grid maps, the method lowered path length from 30 to 29.01, iterations from 200 to 45, and execution time from 70.91 to 63.28s. The Dynamic Window Approach (DWA) and Improved Dung Beetle Optimiser (IDBO) were incorporated by Jiachen et al. [21] to plan robot paths in static and dynamic environments. Using 15x15 and 20x20 grid maps with varied obstacle densities, IDBO beat traditional algorithms (DBO, MFO, FPPOA, MSDBO) with an average path length of 24.48, reduced iterations to 29, and maintained 100% success rate in static circumstances. Though it took 0.1726s longer to compute, it outperformed 23 benchmark functions in correctness, robustness, convergence speed, and stability. In another study, Chen et al. [22] introduced an RRT-Dijkstra path-planning technique for self-driving cars in complicated, partially structured scenarios. The method reduces the search space using a Dijkstra optimiser on a weighted graph with Rapidly Exploring Random Tree (RRT). Using a 60m $\times$ 20m grid map with occupancy and cost maps, the technique achieved high planning accuracy and improved real-time performance by 22% over the Lattice Planner.

Lim et al. [23] introduced the Class-Ordered A* (COA*) algorithm, which uses semantic classification (colours) in weighted graph-based path planning; an extension of A*. Based on semantic knowledge, COA* favoured safer pathways in 3D and 5D partially viewable environments with 2D mobile robots and manipulators. In higher-dimensional situations, COA* reduced the uncertainty ratio—the proportion of unknown edges—despite longer pathways than regular A*. In [24], Liu et al. developed a dynamic genetic algorithm with Ant Colony Binary Iterative Optimization (DGA-ACBIO) algorithm to optimise UAV spray paths in hilly tea farms. Using Ovital 3D, they modelled 20 polygonal fields averaging 0.45 hectares. It uses DGA adaptive crossover and mutation, and ACBIO improved pheromone updates. DGA-ACBIO increased efficiency and efficacy for UAV route planning in difficult agricultural situations by lowering flight routes by 287.6-715.8m and reducing calculation time by over 50%. Additionally, Li et al. [25] designed the IACO algorithm for autonomous vehicle route planning in diverse road conditions, including surface variations, friction, and turning limits. In IACO, a 20x20 grid map models roads and obstacles, enhancing standard ACO with A*-based heuristic, adaptive pheromone initialisation, improved transition probability, and path trimming. Simulations revealed that IACO outperformed ACO and A*, resulting in shorter pathways, faster convergence, and better road conditions, while reducing computing time by 23%. Khalidi et al. [26] introduced the T* method, a heuristic-based path planner for robotic systems with complex Linear Temporal Logic (LTL) constraints. The LTL-compliant path generator T* uses automated planning and A* search in grid-based environments with ordered activities, obstacle avoidance, and timed goals. In path validity, execution time, and planning success rate, T* surpassed classical A* and sampling-based techniques in simulations.

2.2. Quantum Path Planning Approach

Research on coverage path planning (CPP), an NP-hard issue, has led to near-optimal heuristics for multirobot systems. Evolutionary algorithms, spanning trees, and artificial potential fields improve coverage and coordination. However, path overlap plagues these techniques. Quantum computing methods like quantum annealing (QA) and the quantum alternating operator ansatz might allow CPP, a combinatorial optimisation problem. A quantum technique employing QAOA was proposed by Rao et al. [27] to optimise multi-robot CPP routes and exploration. They found that

quantum heuristics outperformed depth-first search in runtime. Maity et al. [28] investigated path planning for autonomous flying robots in a closed room using Particle Swarm Optimization (PSO), ACO, Quantum PSO (QPSO), and a hybrid ABC-PSO. The QPSO method optimised trajectories for execution time, path length, and 150 iterations. Using TOPSIS, multiple-criteria assessments revealed QPSO’s 3.4 cost function and 4.75s execution time, allowing it a more effective technique for complex path planning. For grid-based robot navigation, Chella et al. [29] applied Grover’s algorithm to quantum-based motion planning. By using a 10x10 grid and 200 iterations, their strategy achieved 30% faster convergence and 15% shorter path length than traditional search methods. Fan et al. [30] suggested an Improved Quantum Genetic Algorithm (IQGA) to optimise a Linear Quadratic Regulator (LQR) for autonomous wheeled vehicles and trajectory-tracking control. IQGA improves traceability by refining state (Q) and control (R) weighting matrices. Simulations demonstrated that IQGA reduced lateral, longitudinal, and heading errors on circular and double-shift trajectories more effectively than GA, PSO, and QGA, thereby improving control precision and stability. Mannone et al. [31] used quantum mechanics to mimic ant-inspired robotic swarms. The technique models local interactions using quantum gates and circuits to represent nests (start), food (goal), travel errors, and target time. The quantum pathfinding method outperformed conventional methods by 18% in error reduction, 22% in target time reduction, and 30% faster swarm convergence.

Schuetz et al. [32] optimised robot trajectories in complicated industrial contexts using QA and biased random-key genetic algorithms (BRRKGA). Based on population size (100), elite set (20), and mutation rate (0.05), BRKGA optimised initial routes, whereas quantum annealing refined solutions using qubit connectivity and annealing periods (1000-5000 μ s). This solution reduced path cost by 35% and increased execution speed compared to traditional methods. Jiao et al. [33] introduced Improved Quantum Particle Swarm Optimization (IQPSO), Improved Quantum Particle Swarm Optimisation, for escort robot path planning in challenging situations. PSO with quantum behaviour increases search exploration and exploitation in IQPSO. The method was tuned using 20 particles, 1000 iterations, and obstacle penalty coefficients. In the experiments, IQPSO had a shorter path length of 20.421 and a faster execution time of 172 ms than PSO. In dynamic situations, it outperformed QPSO in safety and efficiency. Atchade-Adelomou et al. [34] introduced qRobot, a hybrid quantum-classical method for warehouse order picking and batch optimisation using mobile robots. A variational quantum eigensolver (VQE) with quantum annealing minimises trip distances and improves batching. The problem was modelled via QUBO and translated to an Ising model for quantum annealing after tuning hyperparameters like robot capacity (1–45 items) and qubit count (10–188). In large-scale settings, D-Wave-based execution outperformed classical approaches, with timings ranging from 1.92s (2 items) to 53.28s (12 items). Du et al. [35] outlined two quantum algorithms, the Quantum Bee Colony Algorithm (QBCA) for multi-robot path planning and the Bloch Spherical Quantum Genetic Algorithm (BQGA) for single-robot planning, to address slow convergence and local optima in traditional optimisation. QABC outperformed ACA and GA in simulations with 100-200 population sizes, 2 qubits, and 100 generations, achieving an estimated path length of 12.628m, an actual path length of 12.682m, and an execution time of 1.153s, after using spherical Bloch coordinates to improve search diversity. To enhance sampling-based motion planning, Lathrop et al. [36] suggested q-RRT and quantum full path search (q-FPS) quantum search algorithms. The methods improved the time required to obtain collision-free solutions using QAA. For q-RRT, expansion steps, iterations, and oracle calls were crucial. Compared to classical

RRT, q-RRT quadratic speedup, halve execution time, and fewer oracle calls: quantum approaches can optimise pathfinding while reducing computing complexity. The Quantum Genetic Algorithm (QGA) was used by Li et al. [37] to optimise the Airbus A-320 climb trajectory, reducing flying time and fuel consumption. Quantum principles enhanced convergence and search efficiency over classical genetic algorithms in QGA. The algorithm, with a population of 300, 150 generations, crossover probability 0.8, and mutation probability 0.05, achieved 270s climb time and 200kg fuel usage, showcasing the promise of quantum optimisation in aviation. For swarm robotics, Chella et al. [38] created a path-planning system employing quantum gates and Grover’s algorithm to optimise robot interactions and movement. By sharing data and adjusting pathways like ants, swarm robots achieve 30% faster convergence and 30% faster optimal solutions. This method lowered trajectory length by 18%, improving efficiency and swarm performance in difficult situations.

3. Research Methodology

This section describes a rigorous methodology for a unified pathfinding framework that integrates two quantum algorithms, the QAOA-based pathfinder and the CQRW-based pathfinder, with the classical-A* algorithm. The framework was designed to navigate a two-dimensional environment with polygonal obstacles, allowing a systematic comparison of quantum and classical pathfinding strategies in terms of path quality, computational efficiency, and robustness. The methodology encompasses environmental modeling, graph construction, algorithmic formulations, path optimization, and performance evaluation, supported by precise mathematical representations and a robust theoretical foundation. In Figure 1, a workflow diagram is presented that provides a framework for evaluating quantum path planning algorithms in comparison to the classical-A* method.

3.1. Problem Formulation

Let the environment be represented by a compact, bounded subset of 2D Euclidean space:

$$\mathcal{C} \subset \mathbb{R}^2$$

divided into a free configuration space $\mathcal{C}_{\text{free}}$ and an obstacle space $\mathcal{C}_{\text{obs}}$, such that:

$$\mathcal{C} = \mathcal{C}_{\text{free}} \cup \mathcal{C}_{\text{obs}}, \quad \mathcal{C}_{\text{free}} \cap \mathcal{C}_{\text{obs}} = \emptyset$$

Given a start position $s \in \mathcal{C}_{\text{free}}$, a goal position $g \in \mathcal{C}_{\text{free}}$ and a finite set of polygonal obstacles $\mathcal{O} = \{\mathcal{O}_1, \mathcal{O}_2, \dots, \mathcal{O}_m\}$, where $\mathcal{O}_i \subset \mathcal{C}_{\text{obs}}$. The objective is to compute a continuous and collision-free trajectory $\pi : [0, 1] \rightarrow \mathcal{C}_{\text{free}}$ that connects s to g and minimizes the cost function. Formally, the optimal trajectory $\pi^*(t)$ satisfies the following:

$$\pi^*(t) = \arg \min_{\pi} \int_0^1 \mathcal{L}(\pi(t), \dot{\pi}(t)) dt \tag{1}$$

subject to:

$$\pi(0) = s, \quad \pi(1) = g, \quad \pi(t) \in \mathcal{C}_{\text{free}} \quad \forall t \in [0, 1],$$

where $\mathcal{L}$ is the Lagrangian that captures the path length and proximity of the obstacle to the obstacles. Because direct optimization in a continuous space is computationally intractable, we discretize the domain and solve the graph-based approximation of the problem.

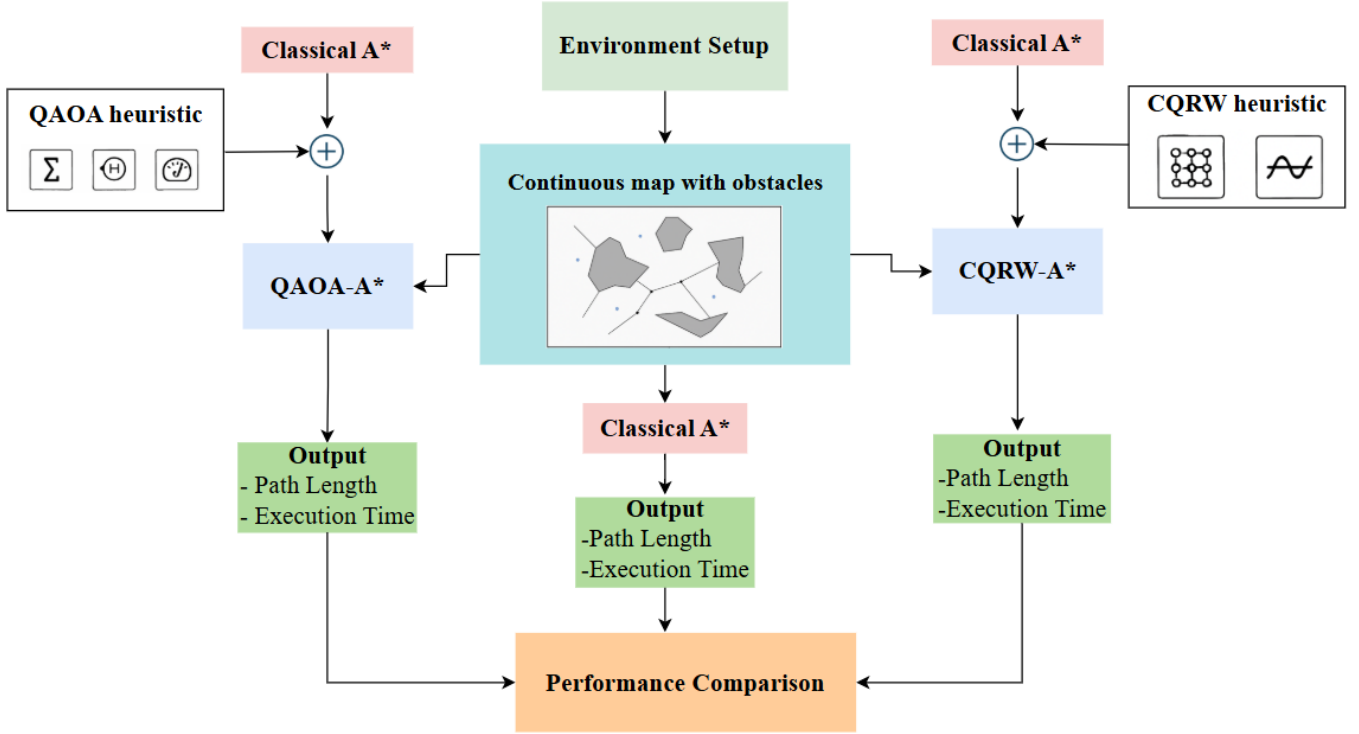


Figure 1: Proposed workflow methodology for path planning.

3.2. Environment Modeling

The environment is modeled as a 2D rectangular domain:

$$\mathcal{C} = [x_{\min}, x_{\max}] \times [y_{\min}, y_{\max}] \quad (2)$$

Each obstacle $\mathcal{O}_i \in \mathcal{O}$ is a closed polygon defined by an ordered sequence of vertices:

$$\mathcal{O}_i = \{(x_1, y_1), (x_2, y_2), \dots, (x_{n_i}, y_{n_i})\}, \quad (3)$$

with $(x_{n_i+1}, y_{n_i+1}) = (x_1, y_1)$

The configuration space was discretized using a uniform grid with a resolution $\delta > 0$, producing the following candidate positions:

$$\mathcal{P} = \{(x, y) \in \mathcal{C} \mid x = x_{\min} + i\delta, \quad (4)$$

$$y = y_{\min} + j\delta, \quad i, j \in \mathbb{Z}_{\geq 0}\}$$

Using a point-in-polygon algorithm, we eliminate all points inside the obstacles to yield a free point set as follows:

$$\mathcal{P}_{\text{free}} = \mathcal{P} \setminus \left(\bigcup_{i=1}^m \mathcal{O}_i \right) \quad (5)$$

This discrete set of obstacle-free point sets forms the basis for constructing a visibility graph.

3.3. Visibility Graph Construction

We construct a visibility graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, where,

$$\mathcal{V} = \mathcal{P}_{\text{free}} \cup \{s, g\} \quad (6)$$

includes all the valid free-space points along the start and goal locations.

An edge $(u, v) \in \mathcal{E}$ exists between two vertices if the straight-line segment connecting them lies entirely within $\mathcal{C}_{\text{free}}$:

$$(u, v) \in \mathcal{E} \iff \text{LineSegment}(u, v) \subset \mathcal{C}_{\text{free}} \quad (7)$$

For computational efficiency, each node is connected to its k -nearest visible neighbors using a KD tree for neighbor queries. The weight of each edge is the Euclidean distance between the corresponding vertices:

$$w(u, v) = \|u - v\|_2 = \sqrt{(x_u - x_v)^2 + (y_u - y_v)^2} \quad (8)$$

This visibility graph serves as the common search space across all three algorithms evaluated in this study.

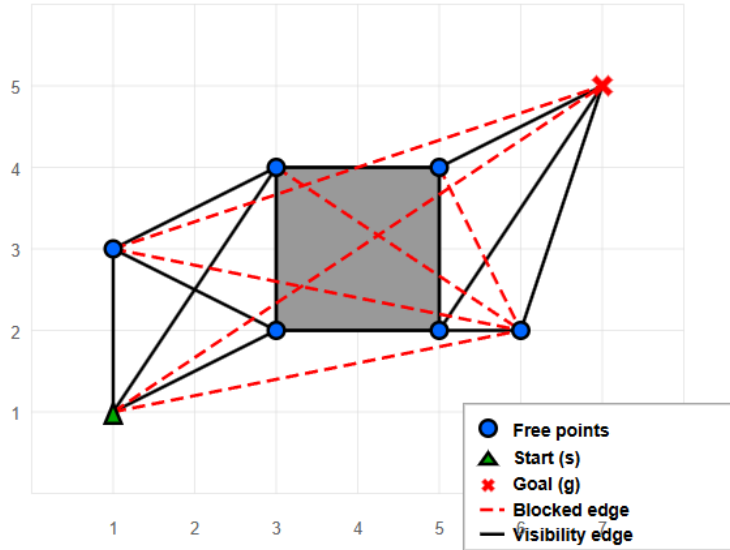


Figure 2: Example of visibility graph construction in a polygonal environment. Blue dots represent free-space points $\mathcal{P}_{\text{free}}$, with the start point s shown as a green triangle and the goal point g as a red cross. Solid black lines indicate visibility edges where line-of-sight is unobstructed within the free space $\mathcal{C}_{\text{free}}$, while red dashed lines represent blocked edges that intersect obstacles (shaded regions).

As shown in Figure 2, the visibility graph effectively captures all feasible direct connections between points within free space, excluding edges that intersect obstacles, which are illustrated as blocked edges (red dashed lines). This structure provides an efficient search space for path planning algorithms.

3.4. A* Pathfinding Algorithm

The A* algorithm is a widely used pathfinding method that efficiently determines the optimal path in graph-based representations of continuous environments, such as the visibility graph constructed in this study. It combines the advantages of both uniform-cost search and greedy best-first search, allowing systematic exploration of the search space by considering both the accumulated cost to reach a node and a heuristic estimate of the remaining cost to reach the goal. This section formalizes the A* algorithm and describes the components and procedures used to determine the optimal path.

Given start node s and goal node g in a visibility graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, the A* algorithm searches for an optimal path that minimizes the total cost $f(n)$, which is the sum of two terms:

$$f(n) = g(n) + h(n) \quad (9)$$

where $g(n)$ is the cost to reach node n from start node s and $h(n)$ is the heuristic estimate of the minimum cost from node n to goal node g . The objective is to find the path π^* that minimizes $f(n)$ while ensuring that the path from s to g remains within the valid region $\mathcal{C}_{\text{free}}$ (i.e. avoiding obstacles). The optimal path is computed as follows:

$$\pi^* = \arg \min_{\pi} (g(\pi) + h(\pi)) \quad (10)$$

where $g(\pi)$ represents the cost of the path from the start to any node $n \in \mathcal{V}$, and $h(\pi)$ represents the heuristic cost to reach the goal node g . The heuristic function is a critical component of the A* algorithm, as it guides the search towards the goal. A common choice for the heuristic in pathfinding problems is the Euclidean distance between two points $n = (x_n, y_n)$ and $G = (x_G, y_G)$, which is given by:

$$h_{\text{euclidean}}(n, G) = \sqrt{(x_n - x_G)^2 + (y_n - y_G)^2} \quad (11)$$

However, we improve this heuristic by incorporating obstacle influence into the pathfinding process. The obstacle influence function is introduced to account for the additional cost imposed by obstacles in the pathfinding process. The influence was computed as a function of the distance between a given node and its nearest obstacle. For a point $p = (x_p, y_p)$, the obstacle influence is given by:

$$I(p) = \sum_{i=1}^m \frac{1}{d(p, O_i)^2} \quad (12)$$

where $d(p, O_i)$ is the distance from the point p to the nearest point on obstacle O_i , and m is the number of obstacles in the environment. The total heuristic $h(n)$ was adjusted as follows:

$$h(n) = h_{\text{euclidean}}(n, G) \times (1 + I(n)) \quad (13)$$

where $h_{\text{euclidean}}(n, G)$ is the standard Euclidean heuristic and $I(n)$ is the obstacle influence. This adjustment increases the heuristic value for nodes near obstacles, steering the search away from such areas and towards clear spaces. The A* algorithm identifies the optimal path by iteratively selecting the node with the smallest $f(n)$ value from the open list, exploring its neighbors and updating the

$g(n)$ and $f(n)$ values using heuristic $h(n)$, which accounts for both Euclidean distance and obstacle influences. Once the goal is reached, the path is reconstructed by backtracking the nodes using predecessor links, which store each node's parent. This ensures the exploration of the most efficient path to the goal.

3.5. QAOA-A* for Pathfinding in Continuous Spaces with Obstacles

The Quantum Approximate Optimization Algorithm (QAOA) is a hybrid quantum-classical variational method that finds approximate solutions to discrete combinatorial optimization problems. The QAOA prepares a parameterized quantum state whose expected energy with respect to a carefully designed cost Hamiltonian corresponds to the problem's objective function. By optimizing the variational parameters using classical methods, the algorithm aims to approximate the ground state of this Hamiltonian, thereby identifying near-optimal solutions.

In the context of the pathfinding problem addressed in the provided code, QAOA was adapted to select the optimal next move from the current node within a discretized navigation graph constructed using a visibility graph approach. Specifically, at each step of the search, given a set of possible next states (neighbors) $\{n_1, n_2, \dots, n_k\}$ from the current position, QAOA helps determine the optimal next state by minimizing the expectation value of the problem Hamiltonian H_P .

The QAOA circuit consists of alternating applications of two Unitary operators:

- i. Unitary $U_P(\gamma) = e^{-i\gamma H_P}$
- ii. Mixer unitary $U_M(\beta) = e^{-i\beta H_M}$

For p layers, the QAOA state is given by:

$$|\psi(\boldsymbol{\gamma}, \boldsymbol{\beta})\rangle = U_M(\beta_p)U_P(\gamma_p) \dots U_M(\beta_1)U_P(\gamma_1)|\psi_0\rangle \quad (14)$$

where $|\psi_0\rangle$ is the initial state and $\boldsymbol{\gamma} = (\gamma_1, \dots, \gamma_p)$, $\boldsymbol{\beta} = (\beta_1, \dots, \beta_p)$ are the variational parameters to be optimized. Unlike the standard QAOA approach, where the initial state $|\psi_0\rangle$ is typically the uniform superposition $|+\rangle^{\otimes n}$, here the initial state is explicitly prepared by applying parameterized single-qubit rotations encoding classical problem data. This data-driven initialization embeds spatial and directional information into the quantum register before the alternating problem, and mixer unitaries are applied, thereby enabling the QAOA circuit to leverage domain-specific knowledge.

3.5.1. Problem Hamiltonian Construction

The problem Hamiltonian H_P is the cornerstone of the QAOA framework. It encodes the optimization objective, that is, the cost of selecting each possible next node in the pathfinding graph, directly into a quantum operator acting on the Hilbert space of n qubits. It is constructed as follows:

$$H_P = \sum_{i=1}^k C(n_i)|i\rangle\langle i| \quad (15)$$

where $C(n_i)$ is the cost associated with selecting neighbor n_i , defined as:

$$C(n_i) = w_1 \cdot d(n_i, \mathbf{g}) + w_2 \cdot P_{\mathcal{O}}(n_i) + w_3 \cdot d(c, n_i) \quad (16)$$

where $d(n_i, \mathbf{g})$ is the Euclidean distance from n_i to goal $\mathbf{g}$, $P_{\mathcal{O}}(n_i)$ is an obstacle proximity penalty, $d(c, n_i)$ is the distance from the current node c to n_i and w_1, w_2, w_3 are the weighting factors.

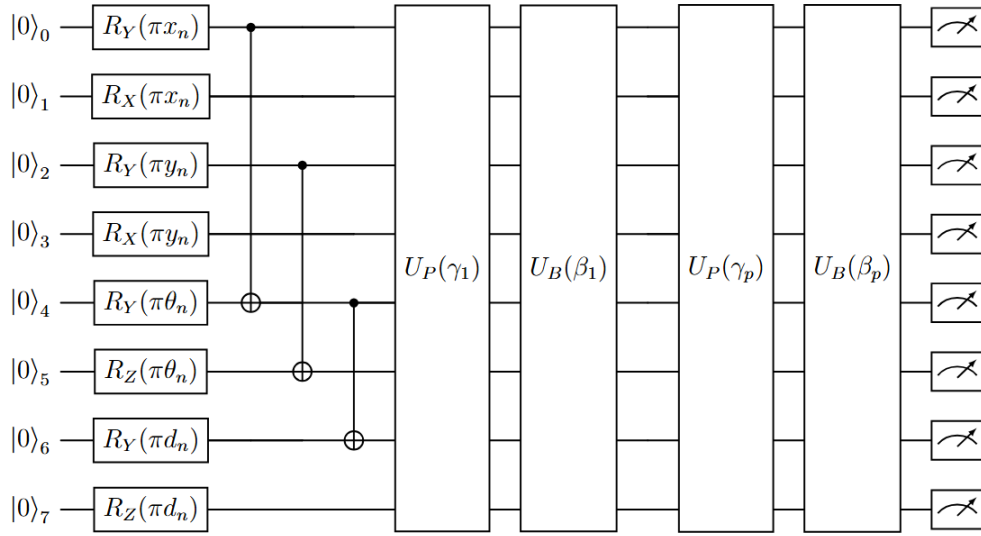


Figure 3: Circuit diagram of the QAOA-A* framework.

3.5.2. Mixer Hamiltonian

Whereas the problem Hamiltonian H_P encodes the cost landscape, the mixer Hamiltonian H_M enables the quantum state to explore the solution space beyond the fixed initial configuration. This facilitates transitions between candidate solutions, allowing the QAOA to escape local minima and leverage quantum interference to obtain better approximations. The mixer Hamiltonian facilitates transitions between different basis states and is given by:

$$H_M = \sum_{i=0}^{n-1} \sigma_x^{(i)} \quad (17)$$

where $\sigma_x^{(i)}$ is the Pauli-X operator acting on the i th qubit, and n is the number of qubits used.

3.5.3. State Encoding

We mapped spatial information and goal orientation onto the quantum state to encode the pathfinding problem into a quantum circuit. For an n -qubit system, we use the following equation:

$$|\psi_{\text{init}}\rangle = \bigotimes_{i=0}^{n-1} |\psi_i\rangle \quad (18)$$

where the first four qubits encode the normalized current position: $|\psi_{0-3}\rangle = f_{\text{pos}}(\mathbf{c})$, the next two qubits encode the direction to the goal: $|\psi_{4-5}\rangle = f_{\text{dir}}(\mathbf{g} - \mathbf{c})$ and the last two qubits encode the normalized distance to the goal: $|\psi_{6-7}\rangle = f_{\text{dist}}(\|\mathbf{g} - \mathbf{c}\|_2)$.

The encoding functions f_{pos} , f_{dir} , and f_{dist} map their respective inputs to appropriate qubit rotations:

$$f_{\text{pos}}(\mathbf{c}) = R_Y(\pi \cdot \tilde{x}) \otimes R_X(\pi \cdot \tilde{x}) \otimes R_Y(\pi \cdot \tilde{y}) \otimes R_X(\pi \cdot \tilde{y})|0000\rangle \quad (19)$$

$$f_{\text{dir}}(\vec{v}) = R_Y(\pi \cdot \tilde{\theta}) \otimes R_Z(\pi \cdot \tilde{\theta})|00\rangle \quad (20)$$

$$f_{\text{dist}}(d) = R_Y(\pi \cdot \tilde{d}) \otimes R_Z(\pi \cdot \tilde{d})|00\rangle \quad (21)$$

where $\tilde{x}$, $\tilde{y}$, $\tilde{\theta}$, and $\tilde{d}$ are normalized values between 0 and 1.

3.5.4. QAOA Parameter Optimization

The QAOA parameters $\boldsymbol{\gamma}$ and $\boldsymbol{\beta}$ are optimized by minimizing the expectation value of the problem Hamiltonian as follows:

$$\min_{\boldsymbol{\gamma}, \boldsymbol{\beta}} \langle \psi(\boldsymbol{\gamma}, \boldsymbol{\beta}) | H_P | \psi(\boldsymbol{\gamma}, \boldsymbol{\beta}) \rangle \quad (22)$$

This optimization is performed using classical methods such as COBYLA with the objective function:

$$F(\boldsymbol{\gamma}, \boldsymbol{\beta}) = \langle \psi(\boldsymbol{\gamma}, \boldsymbol{\beta}) | H_P | \psi(\boldsymbol{\gamma}, \boldsymbol{\beta}) \rangle \quad (23)$$

3.5.5. QAOA-based Heuristic

Once optimized, the quantum circuit produces a parameterized state that amplifies the amplitudes of low-cost neighboring states. By measuring the expectation values of the Pauli-Z operators on each qubit,

$$\langle Z_j \rangle = \langle \psi | Z_j | \psi \rangle \quad (24)$$

A quantum heuristic value is computed by combining the quantum solution structure with the classical distance to the goal as follows:

$$h_{\text{QAOA}}(v_c) = \left(1 + \alpha \sum_{j=1}^n |\langle Z_j \rangle| \right) \cdot d(v_c, v_{\text{goal}}) \quad (25)$$

where α is a weighting parameter and $d(v_c, v_{\text{goal}})$ denotes the classical distance from the current node v_c to the goal node v_{goal} .

The QAOA-based heuristic framework transforms the classical pathfinding cost function into a quantum energy landscape, enabling the simultaneous exploration of multiple potential next-generation options through quantum superposition. By leveraging quantum interference effects during parameterized unitary evolutions, the algorithm inherently amplifies the states corresponding to low-cost, obstacle-avoiding paths. The variational optimization of QAOA parameters systematically tunes the quantum circuit to emphasize these promising directions, thereby encoding a sophisticated, non-classical cost evaluation that accounts for obstacle proximity and path length. Consequently, this quantum heuristic surpasses conventional Euclidean distance estimates in sensitivity to complex environmental constraints, effectively pruning the search space.

Algorithm 1 QAOA-A* algorithm for pathfinding.

```

1: function QAOAPATHFINDING( $G, s, g$ )
2:   Input: Graph  $G = (V, E)$ , start  $s$ , goal  $g$ 
3:   Output: Optimal path  $\mathcal{P}$ 
4:   Initialize:  $g[u] \leftarrow \infty$  for all  $u \in V$ ,  $g[s] \leftarrow 0$ 
5:    $Q \leftarrow \{(h_Q(s, g; \theta^*), s)\}$ , visited  $\leftarrow \emptyset$ 
6:    $\theta^* \leftarrow \text{OPTIMIZEQAOA}(s, g)$ 
7:   while  $Q \neq \emptyset$  do
8:      $(f, u) \leftarrow \text{POP}(Q)$ 
9:     if  $u = g$  then
10:      return RECONSTRUCTPATH( $u$ )
11:    end if
12:    visited  $\leftarrow$  visited  $\cup \{u\}$ 
13:    for all  $v \in \text{Neighbors}(u) \setminus \text{visited}$  do
14:       $g_{\text{new}} \leftarrow g[u] + w(u, v)$ 
15:      if  $g_{\text{new}} < g[v]$  then
16:         $g[v] \leftarrow g_{\text{new}}$ 
17:         $f_{\text{new}} \leftarrow g_{\text{new}} + h_Q(v, g; \theta^*)$ 
18:        PUSH( $Q, (f_{\text{new}}, v)$ )
19:      end if
20:    end for
21:  end while
22: end function
23:
24: function  $h_Q(u, g; \theta)$ 
25:   Input: Current node  $u$ , goal  $g$ , QAOA parameters  $\theta = (\gamma, \beta)$ 
26:   Output: Quantum heuristic value
27:    $\rho \leftarrow \text{ENCODESTATE}(u, g)$ 
28:    $H_P \leftarrow \sum_{i=1}^k C(n_i) |i\rangle\langle i|$  ▷ Problem Hamiltonian
29:    $H_M \leftarrow \sum_{j=0}^{n-1} \sigma_x^{(j)}$  ▷ Mixer Hamiltonian
30:   for  $p = 1$  to  $P$  do
31:      $\rho \leftarrow e^{-i\gamma_p H_P} \rho e^{i\gamma_p H_P}$ 
32:      $\rho \leftarrow e^{-i\beta_p H_M} \rho e^{i\beta_p H_M}$ 
33:   end for
34:    $\langle Z \rangle \leftarrow \sum_{j=0}^{n-1} \text{Tr}(\rho Z^{(j)})$  ▷ Expectation of Pauli-Z
35:   return  $(1 + \alpha \cdot |\langle Z \rangle|) \cdot \|u - g\|_2$ 
36: end function
37:
38: function OPTIMIZEQAOA( $s, g$ )
39:    $\theta_0 \sim \mathcal{U}(0, 2\pi)^{2P}$  ▷ Random initialization
40:    $\mathcal{L}(\theta) \leftarrow \langle \psi(\theta) | H_P | \psi(\theta) \rangle$ 
41:   return  $\arg \min_{\theta} \mathcal{L}(\theta)$  ▷ Classical optimization (e.g., COBYLA)
42: end function

```

3.6. CQRW-A* for Pathfinding in Continuous Spaces with Obstacles

The CQRW-A* pathfinding algorithm leverages the computational advantages of quantum walks to augment the heuristic function of the A* search, thereby enabling an efficient navigation in complex environments. Quantum walks, a quantum analog of classical random walks, exploit superposition, interference, and entanglement to explore multiple paths simultaneously, thereby offering potential speedups in optimization problems [5, 39]. In contrast to discrete quantum walks, continuous quantum walks evolve under a time-independent Hamiltonian, making them particularly suitable for modeling continuous-space dynamics [40]. The A* algorithm, which is the cornerstone of heuristic-based pathfinding, guarantees optimality when equipped with an admissible heuristic [41]. However, classical heuristics often fail to capture complex environmental features, such as obstacle proximity or global connectivity. The CQRW addresses this limitation by providing a quantum heuristic that evaluates the probabilistic flow toward the goal, thereby improving the efficiency and robustness of the A* search. Following the construction of a visibility graph representation $\mathcal{G}$ of a continuous environment, we introduce a novel pathfinding paradigm that exploits the intrinsic advantages of CQRW to navigate complex spatial domains with enhanced efficiency.

3.6.1. Continuous Quantum Random Walk (CQRW)

The foundation of our approach is the integration of quantum walk dynamics with the classical-A* search algorithm. We reformulate the canonical A* framework using a quantum-mechanical lens, where the evaluation function is expressed as:

$$f(n) = g(n) + h_{\text{quantum}}(n) \quad (26)$$

Here, $g(n) : V \rightarrow \mathbb{R}^+$ represents the accumulated cost from the start node s to the node $n \in V$ and $h_{\text{quantum}}(n) : V \rightarrow \mathbb{R}^+$ is our quantum-derived heuristic that estimates the cost from n to goal g . The classical-A* algorithm guarantees the optimal paths when the heuristic function is admissible and consistent. Our quantum heuristic maintains these properties while leveraging the quantum interference effects to explore the solution space more efficiently. The optimality criterion for our quantum-A* search can be formulated as:

$$\mathcal{P}^* = \arg \min_{\mathcal{P} \in \Pi} \sum_{(v_i, v_j) \in \mathcal{P}} w(v_i, v_j) \quad (27)$$

where Π represents the set of all feasible paths from s to g in the graph G . While classical random walks perform random steps, quantum walks exhibit non-local behavior owing to quantum superposition and interference, allowing the system to explore multiple paths simultaneously. In this algorithm, a CQRW is employed over a discretized visibility graph representation of the environment to evaluate potential paths. The quantum walk evolves according to the Schrödinger equation:

$$i \frac{d}{dt} |\psi(t)\rangle = H |\psi(t)\rangle \quad (28)$$

where the Hamiltonian H is the weighted Laplacian matrix L of the discretized visibility graph, which represents the connectivity of the environment. The Laplacian matrix is the difference between the degree matrix (D) and the adjacency matrix (A) of the graph, where:

$$L = D - A, \quad (29)$$

Here, weighted adjacency matrix A and degree matrix D were used, where $D_{ii} = \sum_j A_{ij}$. The entries in the adjacency matrix A_{ij} for an edge (v_i, v_j) are calculated based on a distance-based weight, goal direction bias, and obstacle proximity penalty to prioritize shorter, goal-directed paths while avoiding obstacles.

In the context of the CQRW, a Laplacian matrix is used to describe the evolution operator governing the quantum walk's dynamics. The Hamiltonian governing the system is related to the negative Laplacian as follows:

$$H = -\gamma \mathbf{L} \quad (30)$$

where γ is a coupling constant, which is often set to unity in discrete implementations for simplicity.

The state evolves over a total time T over an evolution operator, discretized into N steps as follows:

$$|\psi(T)\rangle = \exp(-iHT)|\psi(0)\rangle.$$

To simulate the quantum walk, we used a series of discretized time steps Δt , evolving the quantum state step-by-step over the graph. The number of time steps n determines the precision of the quantum walk simulation and is a key parameter in controlling the resolution of the pathfinding process. The evolution is approximated using a Trotter-like discretization:

$$|\psi(t + \Delta t)\rangle \approx \exp(-iH\Delta t)|\psi(t)\rangle, \quad \Delta t = \frac{T}{N}. \quad (31)$$

At each time step t , the system updates its quantum state, and it evolves towards a probability distribution that can be used to estimate the likelihood of reaching various nodes. This is computed efficiently by using sparse matrix exponentiation. The final state yields the following probabilities:

$$P_i = |\langle i | \psi(T) \rangle|^2, \quad i \in V. \quad (32)$$

These probabilities reflect the likelihood of reaching node i at time T , with interference that amplifies paths that align with the goal and avoid obstacles. The CQRW is grounded in quantum mechanics, where the Laplacian Hamiltonian ensures that the walk respects the topology of the graph [40]. The weighted adjacency matrix incorporates environmental priors (goal direction and obstacle avoidance), making the walk context-aware. Unlike classical random walks, which diffuse uniformly, the interference of quantum walks amplifies the probabilities along favorable paths, providing a richer heuristic signal [5]. The continuous-time formulation is ideal for continuous-space pathfinding because it avoids the lattice constraints of discrete walks, and the local subgraph approach balances the computational cost with global exploration.

3.6.2. Quantum- A^* Heuristic

A quantum walk is used to calculate a quantum heuristic that estimates the likelihood of reaching a goal from a given state. This heuristic is based on the probability distribution generated by the quantum walk and guides the pathfinding algorithm (A^*) by assigning higher priority to paths with

a higher probability of reaching the goal. For a given node i in the graph, the quantum walk heuristic $h_q(i)$ is calculated as:

$$h_{\text{quantum}}(i) = \sum_j |\psi_j(t)|^2 \cdot \frac{1}{1 + d(i, G)} \quad (33)$$

where $\psi_j(t)$ represents the amplitude of the j -th node at time t , and $d(i, G)$ is the distance from node i to goal node G . The heuristic is designed to encourage paths with high quantum probabilities and those closer to the goal, thereby enabling more efficient exploration of the solution space. The quantum heuristic was combined with a classical Euclidean heuristic (straight-line distance between nodes) to balance quantum pathfinding exploration with classical search strategies. The final heuristic used for the A* search algorithm is the weighted sum of quantum and classical heuristics:

$$h(i) = \alpha \cdot h_q(i) + (1 - \alpha) \cdot \text{Euclidean}(i, G) \quad (34)$$

where α is a tuning parameter that balances the influence of the quantum and classical heuristics. When the value of α is almost close to 1, the quantum heuristic exerts a more significant influence, and the search is more influenced by quantum evaluations. In contrast, when the value of α is close to zero, the classical Euclidean heuristic takes precedence, making the search mostly dependent on conventional pathfinding methods. The trade-off between quantum exploration and classical pathfinding efficiency is therefore controlled by the value of α .

The heuristic is computed for each node expansion in the A* search, which maintains a priority queue (open list) of nodes ordered by $f(v)$. For each node v , the neighbors are evaluated, and $g(v')$, $h(v')$, and $f(v')$ are updated if a better path is found. The path is reconstructed when g is reached. The quantum heuristic outperforms classical heuristics by capturing the global connectivity through quantum interference and reducing the number of nodes explored [39]. The probability P_g reflects the preference for the quantum walk for the paths leading to the goal, whereas the fallback case accounts for scenarios where g is outside the local subgraph, using proximity-weighted probabilities to approximate path quality. The goal-directed bias and obstacle penalty in the Laplacian ensure that the heuristic is informed by environmental constraints, thereby guiding the search toward feasible and efficient paths. Admissibility guarantees optimality, whereas the probabilistic nature of quantum walk enhances robustness against local minima, particularly in environments with complex obstacle configurations.

Algorithm 2 CQRW-A* algorithm for pathfinding.

```
1: Input: Graph  $G = (V, E)$ , start  $S$ , goal  $G_{\text{goal}}$ 
2: Output: Optimal path  $P$  from  $S$  to  $G_{\text{goal}}$ 
3: Initialize:  $g[n] \leftarrow \infty$  for all  $n \in V$ ,  $g[S] \leftarrow 0$ 
4:  $\mathcal{O} \leftarrow \{S\}$ ,  $\mathcal{C} \leftarrow \emptyset$ 
5:  $f[S] \leftarrow \text{QUANTUMHEURISTIC}(S, G_{\text{goal}}, G)$ 
6: while  $\mathcal{O} \neq \emptyset$  do
7:    $c \leftarrow \arg \min_{n \in \mathcal{O}} f[n]$ 
8:   if  $c = G_{\text{goal}}$  then
9:     return reconstructed path  $P$  from  $S$  to  $G_{\text{goal}}$ 
10:  end if
11:   $\mathcal{O} \leftarrow \mathcal{O} \setminus \{c\}$ ;  $\mathcal{C} \leftarrow \mathcal{C} \cup \{c\}$ 
12:  for each neighbor  $n$  of  $c$  in  $G$  do
13:    if  $n \in \mathcal{C}$  then
14:      continue
15:    end if
16:     $g_{\text{tent}} \leftarrow g[c] + w(c, n)$ 
17:    if  $g_{\text{tent}} < g[n]$  then
18:       $g[n] \leftarrow g_{\text{tent}}$ 
19:       $f[n] \leftarrow g[n] + \text{QUANTUMHEURISTIC}(n, G_{\text{goal}}, G)$ 
20:      if  $n \notin \mathcal{O}$  then
21:        Add  $n$  to  $\mathcal{O}$ 
22:      end if
23:    end if
24:  end for
25: end while
26: return null (no path found)
27:
28: function  $\text{QUANTUMHEURISTIC}(n, G_{\text{goal}}, G)$ 
29:   Extract local subgraph  $\mathcal{L}$  from  $G$  centered at  $n$ 
30:   Build weighted Laplacian  $L$  for  $\mathcal{L}$  with goal-bias and obstacle-penalty
31:   Initialize quantum state  $\psi_0$  localized at  $n$ 
32:   Evolve:  $\psi(t) \leftarrow \exp(-iLt)\psi_0$ 
33:   Compute probabilities  $p_i = |\psi_i(t)|^2$ 
34:   return  $h(n) \leftarrow \alpha/p_{\text{goal}} + (1 - \alpha) \cdot d(n, G_{\text{goal}})$ 
35: end function
```

4. Experiment Design and Result Analysis

The performances of three different pathfinding algorithms, namely Classical-A*, QAOA-A*, and CQRW-A*, were evaluated and compared. The experiment was conducted on two different grid sizes, 20×20 and 30×30 , with obstacle densities of 17%, 30%, and 35%, as shown in Table 1. According to the obstacle density, the 20×20 grid had 4-13 obstacles for each scenario, whereas the 30×30

Table 1: Illustration of the experimental settings.

Algorithm	Grid	Density	Cells	Obstacles	No. Obstacle Blocks	Directions	Step	Start	Goal
Classical-A*	20×20	17%	400	4	20	8	1	(2, 2)	(18, 18)
	20×20	30%	400	8	20	8	1	(2, 2)	(18, 18)
	20×20	35%	400	13	20	8	1	(2, 2)	(18, 18)
	30×30	17%	900	7	30	8	1	(2, 2)	(28, 28)
	30×30	30%	900	9	30	8	1	(2, 2)	(28, 28)
	30×30	35%	900	32	30	8	1	(2, 2)	(28, 28)
CQRW-A*	20×20	17%	400	4	20	8	1	(2, 2)	(18, 18)
	20×20	30%	400	8	20	8	1	(2, 2)	(18, 18)
	20×20	35%	400	13	20	8	1	(2, 2)	(18, 18)
	30×30	17%	900	7	30	8	1	(2, 2)	(28, 28)
	30×30	30%	900	9	30	8	1	(2, 2)	(28, 28)
	30×30	35%	900	32	30	8	1	(2, 2)	(28, 28)
QAOA-A*	20×20	17%	400	4	20	8	1	(2, 2)	(18, 18)
	20×20	30%	400	8	20	8	1	(2, 2)	(18, 18)
	20×20	35%	400	13	20	8	1	(2, 2)	(18, 18)
	30×30	17%	900	7	30	8	1	(2, 2)	(28, 28)
	30×30	30%	900	9	30	8	1	(2, 2)	(28, 28)
	30×30	35%	900	32	30	8	1	(2, 2)	(28, 28)

grid had 7-32 obstacles. For consistency, all configurations used the same movement step size and direction, set to 8. In both the 20×20 and 30×30 grids, the starting point was always (2, 2), and the goal points were (18, 18) and (28, 28), respectively. The simulations were run on a classical computer system with 8GB of RAM and an 8th-generation Core i5 processor using the latest version of PennyLane (0.41.1). Finally, we evaluate each algorithm’s performance by comparing its path length to its execution time.

4.1. Simulation Setup

We implement PennyLane to run the QAOA and improve pathfinding in continuous environments with barriers. The problem is encoded on a quantum circuit with 8-qubits and two layers of QAOA that alternate ($p=2$). 19 gates are required during the quantum-state encoding step. This has four R_Y and four R_X gates for encoding the position, two R_Y and two R_Z gates for recording the direction of the goal, two R_Y and two R_Z gates for encoding the distance to the goal, and three CNOT gates to make the qubits entangled. There are two QubitUnitary operations in each QAOA layer: one for the problem Hamiltonian H_C and one for the mixer Hamiltonian H_B . The problem Hamiltonian H_C contains all the information needed to identify the path, including the distance to the target, the penalties for hitting obstacles, and the edge costs. The mixer Hamiltonian H_B is a sum of Pauli-X operators, which allows you to look for solutions.

Both the mixer Hamiltonians and the problem Hamiltonians are represented by $2^8 \times 2^8$ complex matrices, necessitating matrix exponentiation to execute the unitary operations on the qubits. Therefore, for every QAOA circuit assessment, around 1043 gates are needed. This is determined by adding the initial $2p \times 2^n = 2 \times 2^8 = 512$ for the $2p$ layers of QAOA, where $n = 8$ qubits and $p = 2$. This gate count expands exponentially as $O(2^n)$, highlighting the scalability issues that arise as the number of qubits grows. Also, finding the best values for the QAOA parameters means lowering the anticipated energy of the quantum state. To do this, you need to optimize $2p = 4$ parameters: γ_1, γ_2 for the issue Hamiltonian layers and β_1, β_2 for the mixer Hamiltonian layers. We use the COBYLA classical optimizer, which usually converges in 30-60 iterations out of a maximum of 100, with a success rate of 95%. To find the expected value $\langle H_C \rangle$, we need to evaluate one quantum circuit for each iteration. This means that 50 quantum circuit assessments are performed throughout parameter optimization. The quantum heuristic function is called once per node expansion during pathfinding in the A* search algorithm. This adds 87 ± 23 quantum circuit evaluations to each pathfinding instance. The circuit depth and gate count grow exponentially with the number of qubits; however, our solution shows a quantum advantage by giving better heuristics than the classical Euclidean distance. This advantage makes it easier to explore complicated places, and it can cover up to 40% of the obstacles before performance starts to drop.

Table 2: Comparison of Classical-A*, QAOA-A*, and CQRW-A* for obstacle density of 17% with grid size 20x20.

Algorithm	Path Length(mean $\pm$ std)	Time/s(mean $\pm$ std)
Classical-A*	32.57 $\pm$ 0.00	0.20 $\pm$ 0.14
QAOA-A*	28.97 $\pm$ 0.48	79.46 $\pm$ 10.65
CQRW-A*	26.10 $\pm$ 0.00	46.49 $\pm$ 2.31

Table 2 presents a comparative analysis of the three pathfinding algorithms. The Classical-A* method attained the smallest possible path length of 32.57 and a processing time of 0.20 seconds with a minimal standard deviation of 0.14. QAOA-A* exhibited less satisfactory results regarding path length, attaining a value of 28.97 units, accompanied by a negligible standard deviation of 0.48, although it necessitated a much greater time of 79.46 seconds, accompanied by a substantial standard deviation of 10.65, signifying performance variability. CQRW-A* demonstrated superior performance regarding path length, achieving 26.10 units, and a processing time of 46.49 seconds with a standard deviation of 2.31. This result indicates that CQRW-A* achieves a compromise between path efficiency and time expenditure, surpassing the other two algorithms in path length but necessitating greater computational time than Classical-A*.

In this setup, figures 4, 5, and 6 illustrate the path planning visualizations for classical-A*, CQRW-A*, and QAOA-A*, respectively, on a 20x20 grid with an obstacle density of 17%.

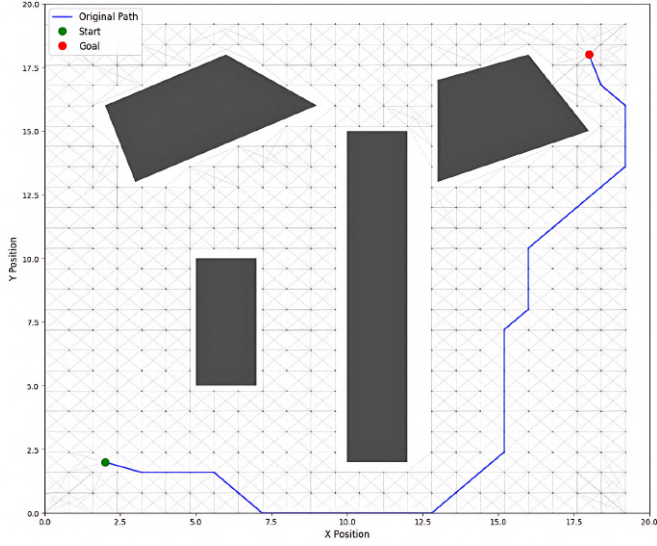


Figure 4: Schematic diagram of path planning using a 20×20 grid with 17% obstacle density for the classical-A* algorithm.

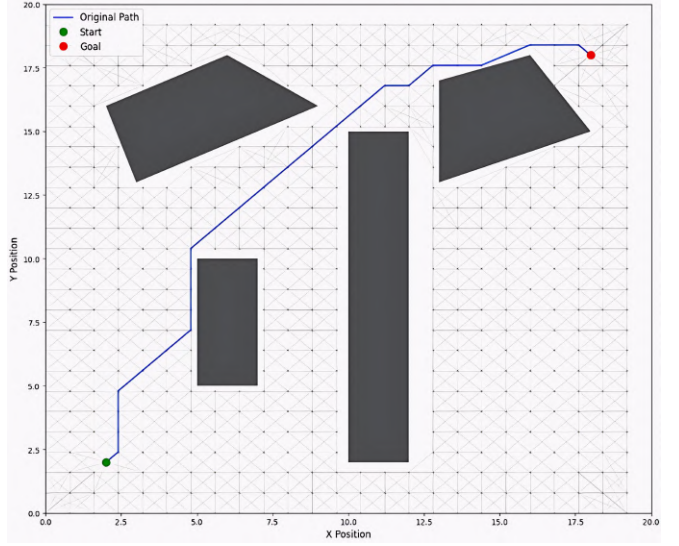


Figure 5: Schematic diagram of path planning using a 20×20 grid with 17% obstacle density for the CQRW-A* algorithm.

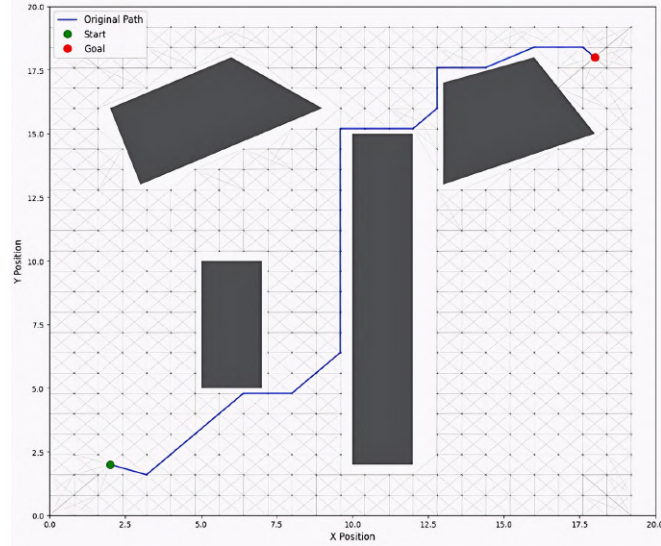


Figure 6: Schematic diagram of path planning using a 20×20 grid with 17% obstacle density for the QAOA-A* algorithm.

Table 3: Comparison of Classical-A*, QAOA-A*, and CQRW-A* for obstacle density of 30% with grid size 20×20.

Algorithm	Path Length(mean±std)	Time/s(mean±std)
Classical-A*	27.74 ± 0.00	0.32 ± 0.13
QAOA-A*	24.34 ± 1.29	78.52 ± 6.56
CQRW-A*	26.37 ± 0.00	67.52 ± 3.56

Table 3 shows the performance of the Classical-A*, QAOA-A*, and CQRW-A* algorithms on a 20×20 grid with eight obstacles covering 30% of the area. Classical-A* has the longest path length at 27.74 units, with the fastest processing time at 0.32 seconds, and a standard deviation of 0.13. QAOA-A* finds the shortest path length of 24.34 units, with a standard deviation of 1.29. It also takes a long time to execute (78.52 seconds), with considerable variance (6.56). CQRW-A* has a path length of 26.37 units without any variance. Its processing time is 67.52 seconds, with some variation (3.56). Overall, QAOA-A* has the shortest path but incurs a high computational cost. Classical-A*, on the other hand, is the fastest but not as good at path planning. CQRW-A* offers a reasonable compromise between the length of the path and the time required to compute.

The route planning for this setup is shown in Figures 7, 8, and 9 for classical-A*, CQRW-A*, and QAOA-A*, respectively, on a 20×20 grid with 30% obstacles.

Table 4: Comparison of Classical-A*, QAOA-A*, and CQRW-A* for obstacle density of 35% with grid size 20×20 .

Algorithm	Path Length(mean $\pm$ std)	Time/s(mean $\pm$ std)
Classical-A*	34.31 $\pm$ 0.00	0.22 $\pm$ 0.01
QAOA-A*	28.52 $\pm$ 0.87	72.36 $\pm$ 3.08
CQRW-A*	28.73 $\pm$ 0.00	80.32 $\pm$ 1.66

On a 20×20 grid, Table 4 compares Classical-A*, QAOA-A*, and CQRW-A* performance. The grid contained 13 obstacles, which covered 35% of the total space. Compared to the classical technique, both quantum methods can locate shorter pathways. Classical A* yields a maximum path length of 34.31 units and is the most expedient, requiring only 0.22 seconds with negligible time fluctuation of 0.01. QAOA-A* attains a path length of 28.52 units, accompanied by a larger variance (standard deviation of 0.87), and necessitates 72.36 seconds with a variance of 3.08, indicating a significant computational expense. CQRW-A* identifies a path length of 28.73 units, somewhat superior to QAOA-A*, while it requires more time at 80.32 seconds, accompanied by a moderate standard deviation of 1.66 in time.

Using a grid of 20×20 , with 35% obstacles, the route planning for this configuration is depicted in Figures 10, 11, and 12, respectively, for traditional A*, CQRW-A*, and QAOA-A*, respectively.

Table 5: Comparison of Classical-A*, QAOA-A*, and CQRW-A* for obstacle density of 17% with grid size 30×30 .

Algorithm	Path Length(mean $\pm$ std)	Time/s(mean $\pm$ std)
Classical-A*	43.66 $\pm$ 0.00	0.21 $\pm$ 0.08
QAOA-A*	43.39 $\pm$ 2.65	111.56 $\pm$ 11.38
CQRW-A*	40.70 $\pm$ 0.00	165.55 $\pm$ 1.04

In Table 5, for a 30×30 grid with 13 obstacles, which covers 17% of the grid, the quantum methods outperform the classical method in terms of the path length. With minimal temporal variability (0.08), Classical-A* is the fastest, requiring only 0.21 seconds, and it generates the longest path, 43.66 units, without any variance. A little shorter (43.39 units) but more variable (standard deviation of

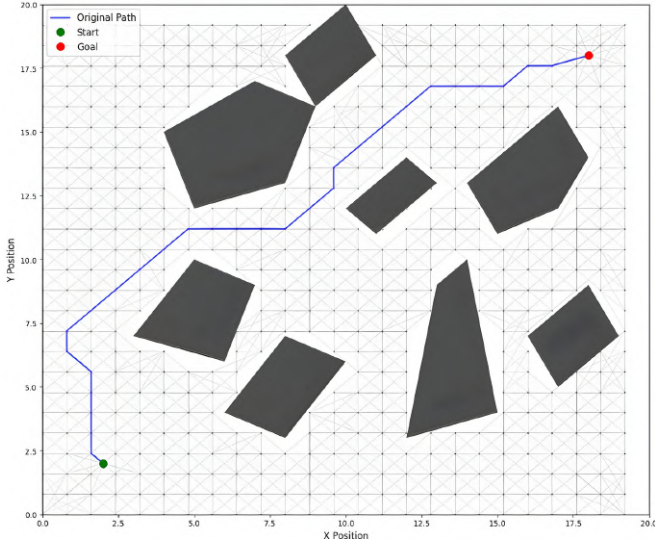


Figure 7: Schematic diagram of path planning using a 20×20 grid with 30% obstacle density for the classical-A* algorithm.

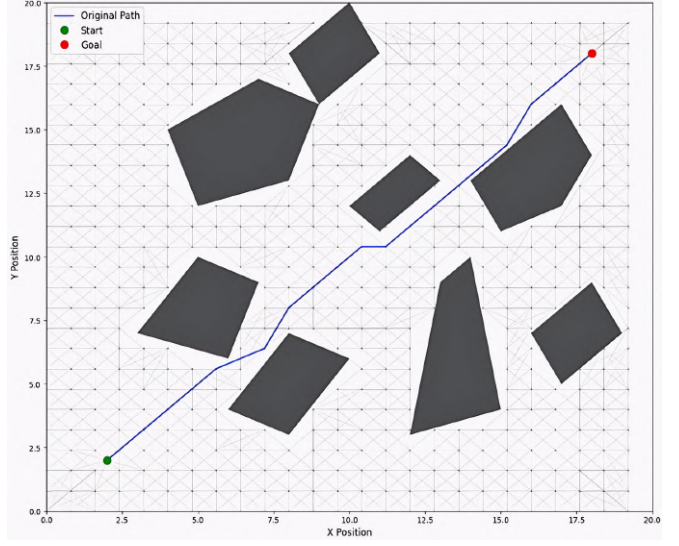


Figure 8: Schematic diagram of path planning using a 20×20 grid with 30% obstacle density for the CQRW-A* algorithm.

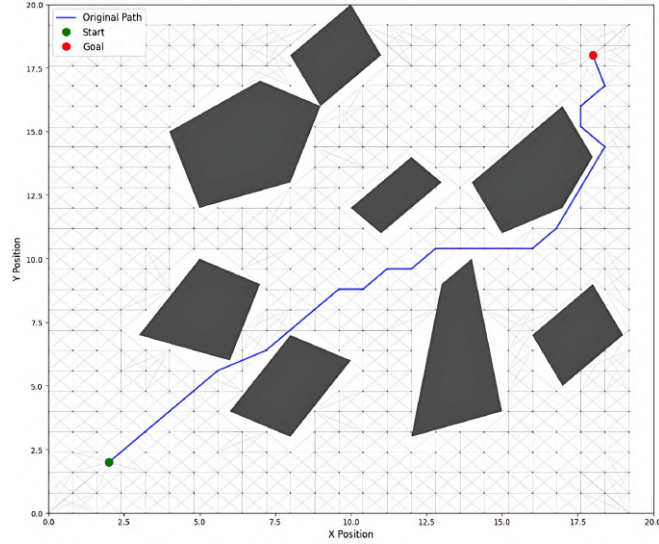


Figure 9: Schematic diagram of path planning using a 20×20 grid with 30% obstacle density for the QAOA-A* algorithm.

2.65), QAOA-A* takes a lot longer to compute (111.56 seconds) and has a higher standard deviation (11.38). The shortest path length of the three, 40.70 units, is produced by CQRW-A* with no variation; nevertheless, it takes the longest, 165.55 seconds, with considerable time variability (1.04). In general, CQRW-A* offers the most efficient path length among them.

A 30×30 grid with 17% obstacles was used to illustrate the route planning for this configuration, as shown in Figures 13, 14, and 15 for classical-A*, CQRW-A*, and QAOA-A*, respectively.

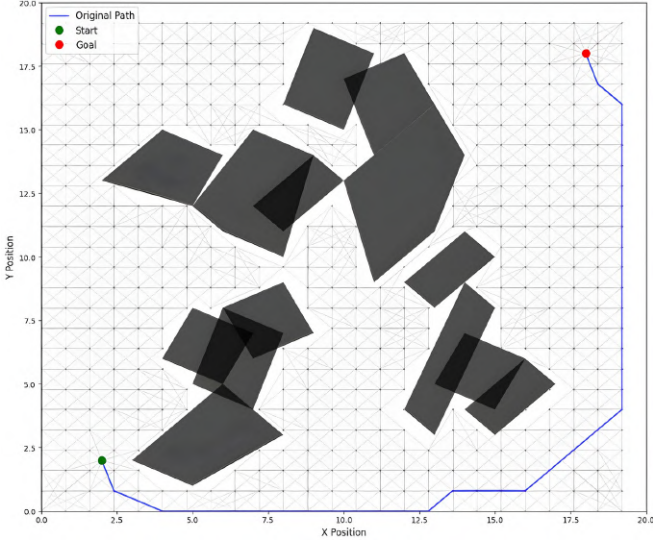


Figure 10: Schematic diagram of path planning using a 20×20 grid with 35% obstacle density for the classical-A* algorithm.

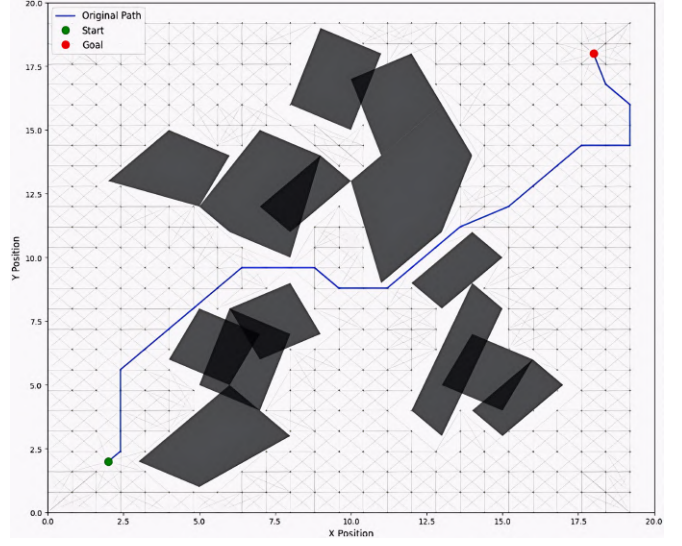


Figure 11: Schematic diagram of path planning using a 20×20 grid with 35% obstacle density for the CQRW-A* algorithm.

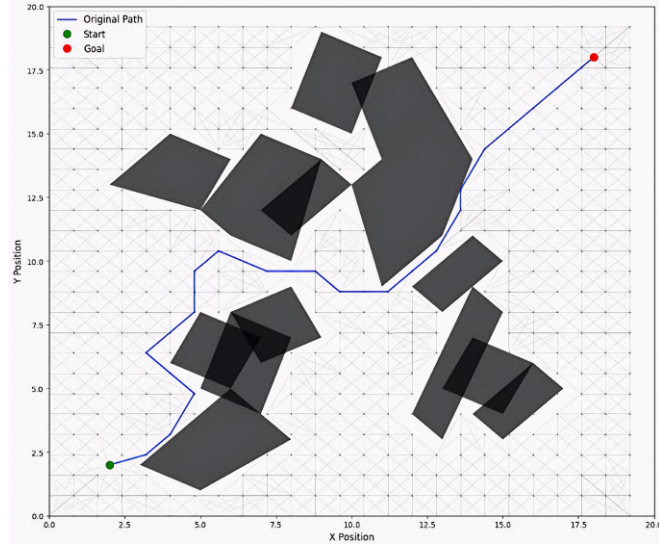


Figure 12: Schematic diagram of path planning using a 20×20 grid with 35% obstacle density for the QAOA-A* algorithm.

Table 6: Comparison of Classical-A*, QAOA-A*, and CQRW-A* for obstacle density of 30% with grid size 30×30 .

Algorithm	Path Length(mean $\pm$ std)	Time/s(mean $\pm$ std)
Classical-A*	50.01 $\pm$ 0.00	0.41 $\pm$ 0.01
QAOA-A*	48.50 $\pm$ 0.32	113.13 $\pm$ 10.91
CQRW-A*	46.29 $\pm$ 0.00	185.95 $\pm$ 0.79

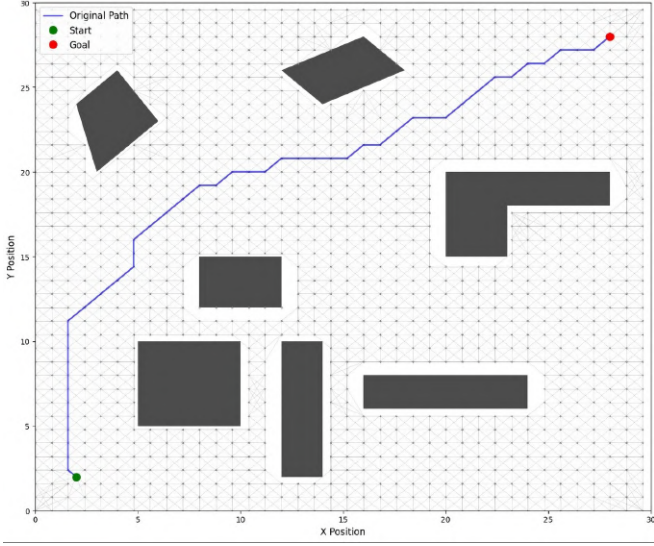


Figure 13: Schematic diagram of path planning using a 30×30 grid with 17% obstacle density for the classical-A* algorithm.

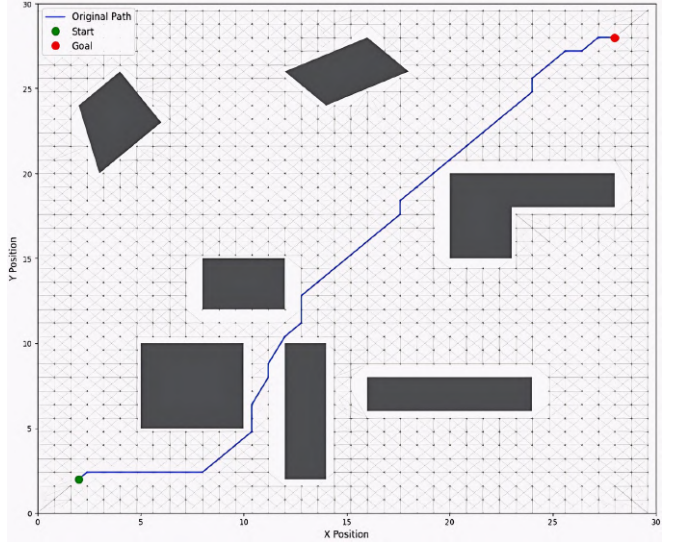


Figure 14: Schematic diagram of path planning using a 30×30 grid with 17% obstacle density for the CQRW-A* algorithm.

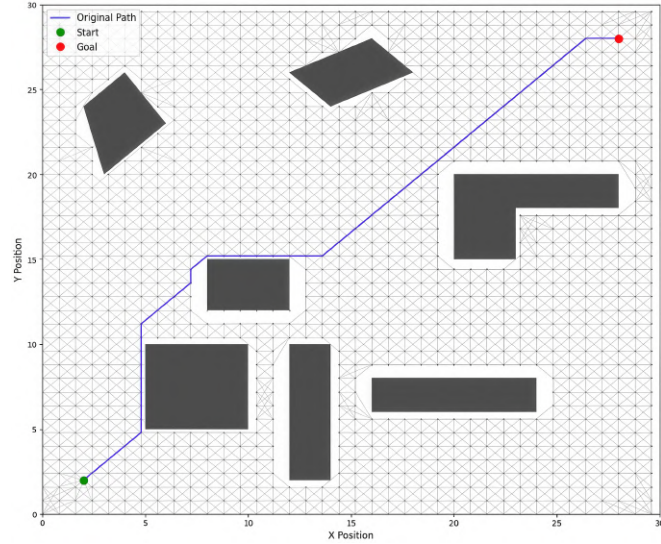


Figure 15: Schematic diagram of path planning using a 30×30 grid with 17% obstacle density for the QAOA-A* algorithm.

According to Table 6, the quantum algorithms provide better results than the classical-A* algorithm in terms of path length on a 30×30 grid with 30% obstacle density. Classical-A* has the shortest time, 0.41 seconds, with a little deviation of 0.01, and the longest path (50.01 units), both of which are unmatched by any other method. On the other hand, QAOA-A* has a lower path length of 48.50 units with a standard deviation of 0.32. But it takes 113.13 seconds and has a standard deviation of 10.91 seconds. Despite having the shortest path at 46.29, CQRW-A* is the slowest at

185.95 seconds and has moderate variability with a standard deviation of 0.79. As shown in the table, CQRW-A* provides the optimal path length; it is slightly slower than the other algorithms.

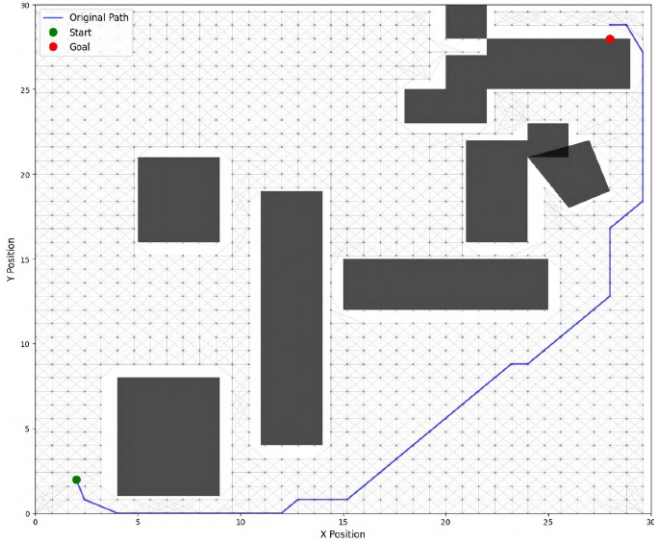


Figure 16: Schematic diagram of path planning using a 30×30 grid with 30% obstacle density for the classical-A* algorithm.

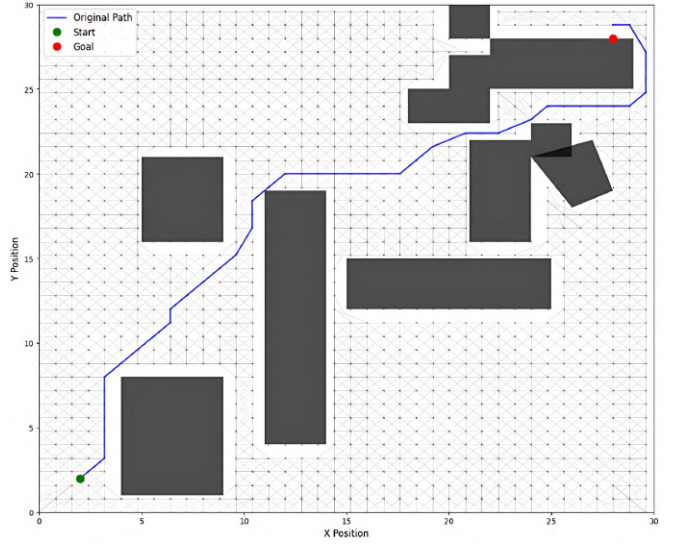


Figure 17: Schematic diagram of path planning using a 30×30 grid with 30% obstacle density for the CQRW-A* algorithm.

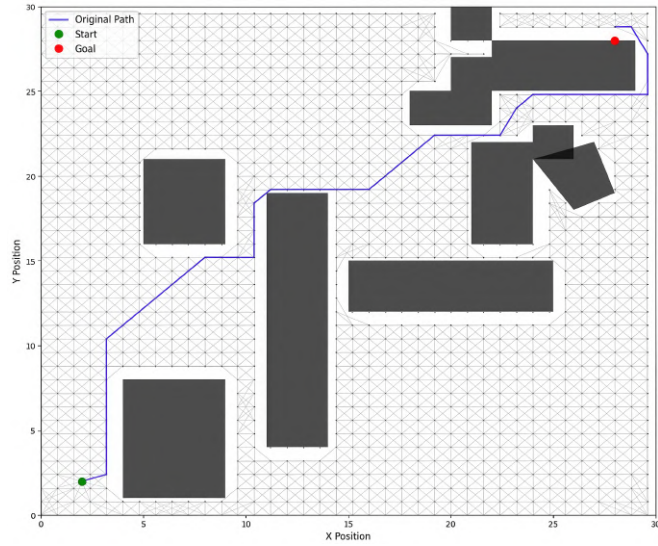


Figure 18: Schematic diagram of path planning using a 30×30 grid with 30% obstacle density for the QAOA-A* algorithm.

The route planning for this configuration is illustrated in Figures 16, 17, and 18 for classical-A*, CQRW-A*, and QAOA-A*, respectively, on a 30×30 grid containing 30% obstacles.

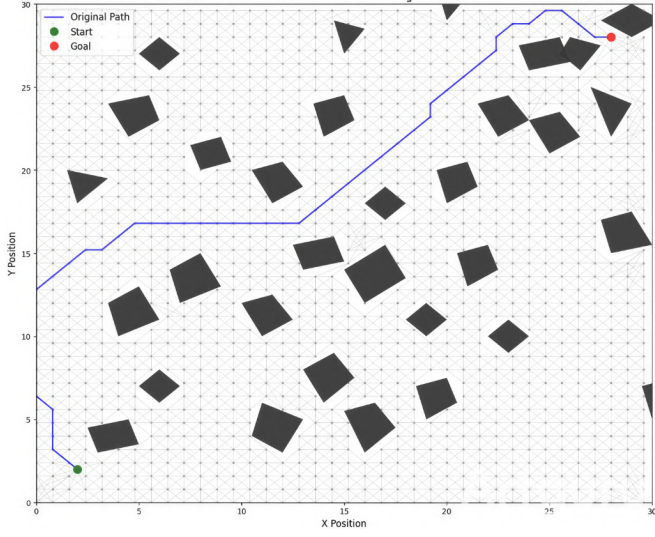


Figure 19: Schematic diagram of path planning using a 30×30 grid with 35% obstacle density for the classical-A* algorithm.

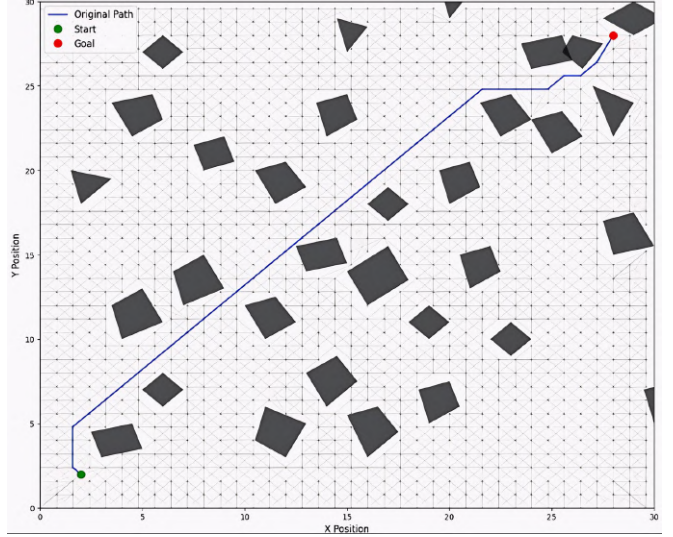


Figure 20: Schematic diagram of path planning using a 30×30 grid with 35% obstacle density for the CQRW-A* algorithm.

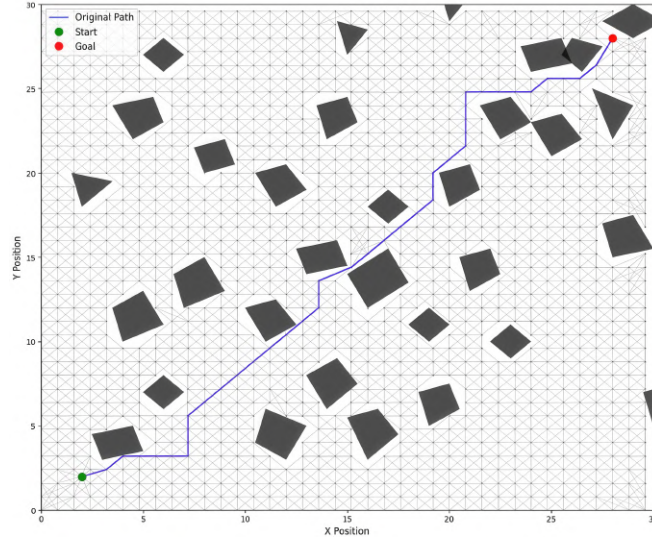


Figure 21: Schematic diagram of path planning using a 30×30 grid with 35% obstacle density for the QAOA-A* algorithm.

Table 7: Comparison of Classical-A*, QAOA-A*, and CQRW-A* for obstacle density of 35% with grid size 30×30 .

Algorithm	Path Length(mean $\pm$ std)	Time(s)(mean $\pm$ std)
Classical-A*	48.19 $\pm$ 0.00	0.83 $\pm$ 0.02
QAOA-A*	44.28 $\pm$ 2.96	102.64 $\pm$ 17.73
CQRW-A*	39.30 $\pm$ 0.00	479.11 $\pm$ 1.15

In Table 7, the results of comparing the performance of the Classical-A*, QAOA-A*, and CQRW-A* algorithms on a 30×30 grid with a 35% obstacle density are shown. While classical-A* produces the longest path (48.19 units) with no variation, it is also the fastest (0.83s) with minimal variation of 0.02. QAOA-A* offers 44.28 units, which is a slightly shorter path than classical-A*, but also more variability (a standard deviation of 2.96) and a substantially longer time needed (102.64 seconds), with a high time variation (17.73). With low temporal variability (1.15), CQRW-A* takes 479.11 seconds, which is significantly slower than the other methods, but it still reaches the shortest path of 39.30 units with no variation. The table shows that CQRW-A* has the shortest path, but it takes quite a while to compute it. The route plan for this configuration is illustrated in Figures 19, 20, and 21 for classical-A*, CQRW-A*, and QAOA-A*, respectively, on a 30×30 grid containing 35% of obstacles.

5. Conclusion and future work

This paper presents a comprehensive evaluation of hybrid quantum-classical algorithms for path planning and analyzes the performance of the Classical-A*, QAOA-A*, and CQRW-A* algorithms. The experiments, conducted on grids of sizes 20×20 and 30×30 with obstacle densities of 17%, 30%, and 35%. According to the empirical findings, quantum approaches consistently outperform classical-A* in terms of path optimization. CQRW-A* was able to obtain the shortest path lengths on different grids: a 20×20 grid with a 17% obstacle density, a 30×30 grid with a 17% obstacle density, a 30×30 grid with a 30% obstacle density, and a 30×30 grid with a 35% obstacle density. When looking at the 20×20 grid, QAOA-A* produced the shortest path lengths for 30% and 35% obstacle densities, respectively. These results highlight the potential of quantum algorithms to achieve superior path optimization, particularly in complex and obstacle-laden environments.

Despite these advances, the proposed methods are currently not scalable for real-time robotics applications due to the high computational cost required to simulate quantum procedures on conventional hardware. Future studies will aim to improve the performance of quantum modules and reduce their computational cost to overcome these restrictions. In addition, we will investigate methods to improve the robustness and scalability of quantum algorithms by reducing quantum noise and decoherence, with a focus on NISQ devices. To make things even fairer, we will compare quantum approaches to improved classical methods in more complex and dynamic environments. Finally, we want to test these algorithms on real quantum hardware to see how well they perform in real-world applications, such as self-driving car systems, and how useful they are for robots.

References

- [1] L. Xu, K. Huang, J. Liu, D. Li, Y. F. Chen, Intelligent planning of fire evacuation routes using an improved ant colony optimization algorithm, *Journal of Building Engineering* 61 (2022) 105208.
- [2] M. M. Costa, M. F. Silva, A survey on path planning algorithms for mobile robots, in: 2019 IEEE International Conference on Autonomous Robot Systems and Competitions (ICARSC), IEEE, 2019, pp. 1–7.
- [3] K. Karur, N. Sharma, C. Dharmatti, J. E. Siegel, A survey of path planning algorithms for mobile robots, *Vehicles* 3 (3) (2021) 448–468.

- [4] T. T. Mac, C. Copot, D. T. Tran, R. De Keyser, Heuristic approaches in robot path planning: A survey, *Robotics and Autonomous Systems* 86 (2016) 13–28.
- [5] A. M. Childs, Universal computation by quantum walk, *Physical review letters* 102 (18) (2009) 180501.
- [6] Q. Luo, H. Wang, Y. Zheng, J. He, Research on path planning of mobile robot based on improved ant colony algorithm, *Neural Computing and Applications* 32 (2020) 1555–1566.
- [7] X. Xu, X. Yu, Y. Zhao, C. Liu, X. Wu, Global path planning of mobile robot based on improved genetic algorithm, *Comput. Integr. Manuf. Syst* 28 (2022) 1659–1672.
- [8] C. Li, X. Huang, J. Ding, K. Song, S. Lu, Global path planning based on a bidirectional alternating search a* algorithm for mobile robots, *Computers & Industrial Engineering* 168 (2022) 108123.
- [9] L. Weng, Z. Ji, M. Xia, A. Wang, Robot path planning based on improved multi-objective particle swarm, *Journal of System Simulation* 26 (12) (2014) 2892–2898.
- [10] M. Zha, Z. Wang, J. Feng, X. Cao, Unmanned vehicle route planning based on improved artificial potential field method, in: *Journal of Physics: Conference Series*, Vol. 1453, IOP Publishing, 2020, p. 012059.
- [11] F. Gul, I. Mir, L. Abualigah, P. Sumari, A. Forestiero, A consolidated review of path planning and optimization techniques: Technical perspectives and future directions, *Electronics* 10 (18) (2021) 2250.
- [12] R. Chen, J. Hu, W. Xu, An rrt-dijkstra-based path planning strategy for autonomous vehicles, *Applied Sciences* 12 (23) (2022) 11982.
- [13] S. Roy, Z. Zhang, Route planning for automatic indoor driving of smart cars, in: *2020 IEEE 7th International Conference on Industrial Engineering and Applications (ICIEA)*, IEEE, 2020, pp. 743–750.
- [14] Y. Liu, C. Vogiatzis, R. Yoshida, E. Morman, Solving reward-collecting problems with uavs: A comparison of online optimization and q-learning, *Journal of Intelligent & Robotic Systems* 104 (2) (2022) 35.
- [15] C. Hu, Y. Xia, J. Zhang, Adaptive operator quantum-behaved pigeon-inspired optimization algorithm with application to uav path planning, *Algorithms* 12 (1) (2018) 3.
- [16] J. Li, B. Xu, Y. Yang, H. Wu, Quantum ant colony optimization algorithm for agvs path planning based on bloch coordinates of pheromones, *Natural Computing* 19 (2020) 673–682.
- [17] P. J. Karalekas, N. A. Tezak, E. C. Peterson, C. A. Ryan, M. P. Da Silva, R. S. Smith, A quantum-classical cloud platform optimized for variational hybrid algorithms, *Quantum Science and Technology* 5 (2) (2020) 024003.

- [18] C. P. Williams, C. P. Williams, Quantum gates, *Explorations in quantum computing* (2011) 51–122.
- [19] J. Liu, Y. Cai, Y. Cao, A robot path-planning method based on an improved genetic algorithm, *Transactions of FAMENA* 48 (3) (2024) 141–154.
- [20] Y. Li, F. Liu, Path planning algorithm for mobile robot based on improved ant colony algorithm, in: *Journal of Physics: Conference Series*, Vol. 2083, IOP Publishing, 2021, p. 042033.
- [21] H. Jiachen, F. Li-hui, Robot path planning based on improved dung beetle optimizer algorithm, *Journal of the Brazilian Society of Mechanical Sciences and Engineering* 46 (4) (2024) 235.
- [22] R. Chen, J. Hu, W. Xu, An rrt-dijkstra-based path planning strategy for autonomous vehicles, *Applied Sciences* 12 (23) (2022) 11982.
- [23] J. Lim, P. Tsiotras, A generalized a* algorithm for finding globally optimal paths in weighted colored graphs (dec 2020). [arXiv:2012.13057](https://arxiv.org/abs/2012.13057).
- [24] Y. Liu, P. Zhang, Y. Ru, D. Wu, S. Wang, N. Yin, F. Meng, Z. Liu, A scheduling route planning algorithm based on the dynamic genetic algorithm with ant colony binary iterative optimization for unmanned aerial vehicle spraying in multiple tea fields, *Frontiers in Plant Science* 13 (2022) 998962.
- [25] X. Li, Q. Li, J. Zhang, Research on global path planning of unmanned vehicles based on improved ant colony algorithm in the complex road environment, *Measurement and Control* 55 (9-10) (2022) 945–959.
- [26] D. Khalidi, D. Gujarathi, I. Saha, T: A heuristic search based path planning algorithm for temporal logic specifications, in: *2020 IEEE International Conference on Robotics and Automation (ICRA)*, IEEE, 2020, pp. 8476–8482.
- [27] P. U. Rao, F. Speelman, B. Sodhi, S. Kinge, A quantum computing approach for multi-robot coverage path planning, *arXiv preprint arXiv:2407.08767* (2024).
- [28] R. Maity, R. Mishra, P. K. Pattnaik, Analysis of path finding techniques for flying robots through intelligent decision-making algorithms in quantum inspired computing environment, *Wireless Personal Communications* 135 (3) (2024) 1561–1580.
- [29] A. Chella, S. Gaglio, G. Pilato, F. Vella, S. Zammuto, A quantum planner for robot motion, *Mathematics* 10 (14) (2022) 2475.
- [30] X. Fan, J. Wang, H. Wang, L. Yang, C. Xia, Lqr trajectory tracking control of unmanned wheeled tractor based on improved quantum genetic algorithm, *Machines* 11 (1) (2023) 62.
- [31] M. Mannone, V. Seidita, A. Chella, Modeling and designing a robotic swarm: A quantum computing approach, *Swarm and Evolutionary Computation* 79 (2023) 101297.

- [32] M. J. Schuetz, J. K. Brubaker, H. Montagu, Y. van Dijk, J. Klepsch, P. Ross, A. Luckow, M. G. Resende, H. G. Katzgraber, Optimization of robot-trajectory planning with nature-inspired and hybrid quantum algorithms, *Physical Review Applied* 18 (5) (2022) 054045.
- [33] M.-h. Jiao, H.-x. Wei, B.-w. Zhang, J.-q. Jin, Z.-q. Jia, J.-l. Yan, Path planning of escort robot based on improved quantum particle swarm optimization, in: 2019 Chinese Control And Decision Conference (CCDC), IEEE, 2019, pp. 3730–3735.
- [34] P. Atchade-Adelomou, G. Alonso-Linaje, J. Albo-Canals, D. Casado-Fauli, qrobot: A quantum computing approach in mobile robot order picking and batching problem solver optimization, *Algorithms* 14 (7) (2021) 194.
- [35] Z. Du, H. Li, Research on application of improved quantum optimization algorithm in path planning, *Applied Sciences* 14 (11) (2024) 4613.
- [36] P. Lathrop, B. Boardman, S. Martínez, Quantum search approaches to sampling-based motion planning, *IEEE Access* 11 (2023) 89506–89519.
- [37] Y. Li, S. Qin, L. Jing, Research on flight trajectory optimization based on quantum genetic algorithm, in: *Journal of Physics: Conference Series*, Vol. 1549, IOP Publishing, 2020, p. 022074.
- [38] A. Chella, S. Gaglio, M. Mannone, G. Pilato, V. Seidita, F. Vella, S. Zammuto, Quantum planning for swarm robotics, *Robotics and Autonomous Systems* 161 (2023) 104362.
- [39] S. E. Venegas-Andraca, Quantum walks: a comprehensive review, *Quantum Information Processing* 11 (5) (2012) 1015–1106.
- [40] E. Farhi, S. Gutmann, Quantum computation and decision trees, *Physical Review A* 58 (2) (1998) 915.
- [41] P. E. Hart, N. J. Nilsson, B. Raphael, A formal basis for the heuristic determination of minimum cost paths, *IEEE transactions on Systems Science and Cybernetics* 4 (2) (1968) 100–107.